Republic of the Philippines
Tarlac State University
COLLEGE OF COMPUTER STUDIES
Tarlac City, Tarlac
Tel. No. (045) 6068173

Simulation of CPU Scheduling Algorithms

A Case Study
Submitted to the Faculty of the College of Computer Studies
San Isidro Campus, Tarlac City,
Tarlac State University

In Partial Fulfillment of the Requirements for the Subject Operating Systems Academic Year 2026-2027

Submitted by:
David, Laurence B.

Subject Instructor:
Jo Anne G. Cura

Date of submission: **May 15, 2026**

1. Introduction

In a multi-programmed operating system, the CPU is a very limited and highly important resource. Since several processes may need the CPU at the same time, the operating system has to decide which process will run next and how long it should execute. This is handled through CPU scheduling, which manages the order of processes in the ready queue so that the CPU is used efficiently while still treating all processes fairly.

This study focuses on creating and implementing a working simulation of different CPU scheduling algorithms. Instead of only studying how they work in theory, a program was built to actually run these algorithms and produce real outputs. The simulation displays a Gantt chart to show the execution flow and also computes important values such as Waiting Time (WT) and Turnaround Time (TAT) for each process. Through these results, it becomes easier to observe and compare how each scheduling method behaves in different situations.

The specific objectives of this program are:

This project presents the implementation and comparison of six fundamental CPU scheduling algorithms: First-Come, First-Served (FCFS), Non-Preemptive Shortest Job First (SJF), Preemptive Shortest Remaining Time (SRT), Round Robin, Priority Scheduling, and a combined Priority with Round Robin method. The system takes user-defined inputs such as arrival time, burst time, priority level, and time quantum to simulate realistic process execution scenarios. It then produces Gantt charts to visualize the scheduling flow and computes key performance measures, including Waiting Time (WT) and Turnaround Time (TAT). The main goal is to assess and compare the efficiency and behavior of preemptive and non-preemptive scheduling techniques based on generated results and average performance metrics.

2. Body

2.1 Implementation Overview

To evaluate these concepts in practice, a Python-based simulator was developed that accepts dynamic inputs such as arrival time, burst time, priority, and time quantum for both preemptive and non-preemptive scheduling. The program is structured around a central Process model that stores important state information, including remaining execution time used for context switching, and it uses a global clock to manage and monitor the ready queue. As the simulation runs, the selected scheduling algorithm decides which process gets CPU access at each moment. This approach produces a clear execution timeline through Gantt charts and also automatically computes key performance metrics like Waiting Time (WT) and Turnaround Time (TAT), allowing for easy comparison between different scheduling methods.

```python
from collections import deque

from copy import deepcopy



class Process:

    def __init__(self, pid, arrival, burst, priority=0):

        self.pid = pid

        self.arrival = arrival

        self.burst = burst

        self.priority = priority

        self.reset()


    def reset(self):

        self.rem = self.burst

        self.finish_time = None

        self.wt = 0

        self.tat = 0



# ================= UTILITIES ================= #


def compute_metrics(p):

    p.tat = p.finish_time - p.arrival

    p.wt = p.tat - p.burst



def print_gantt(gantt):

    print("\nGantt Chart:")

    for pid, s, e in gantt:

        print(f"| {pid} ({s}-{e}) ", end="")

    print("|\n")



def print_metrics(processes):

    processes = sorted(processes, key=lambda x: x.pid)

    total_wt = sum(p.wt for p in processes)

    total_tat = sum(p.tat for p in processes)


    print("PID\tWT\tTAT")

    for p in processes:

        print(f"{p.pid}\t{p.wt}\t{p.tat}")


    print(f"\nAverage WT: {total_wt/len(processes):.2f}")

    print(f"Average TAT: {total_tat/len(processes):.2f}\n")



def clone_processes(processes):

    return deepcopy(processes)



def add_gantt(gantt, pid, start, end):

    if start == end:

        return
```

```
    if gantt and gantt[-1][0] == pid:
        gantt[-1] = (pid, gantt[-1][1], end)
    else:
        gantt.append((pid, start, end))
```

```
# ================= FCFS ================= #

def fcfs(processes):
    procs = sorted(clone_processes(processes), key=lambda x: (x.arrival, x.pid))
    time = 0
    gantt = []

    for p in procs:
        if time < p.arrival:
            add_gantt(gantt, "IDLE", time, p.arrival)
            time = p.arrival

        start = time
        time += p.burst
        p.finish_time = time
        compute_metrics(p)
        add_gantt(gantt, p.pid, start, time)

    print_gantt(gantt)
    print_metrics(procs)
```

```
# ================= SJF ================= #

def sjf(processes):
    procs = clone_processes(processes)
    time = 0
    done = 0
    gantt = []

    while done < len(procs):
        ready = [p for p in procs if p.arrival <= time and p.finish_time is None]

        if not ready:
            next_arrival = min(p.arrival for p in procs if p.finish_time is None)
            add_gantt(gantt, "IDLE", time, next_arrival)
            time = next_arrival
            continue

        p = min(ready, key=lambda x: (x.burst, x.arrival, x.pid))
        start = time
        time += p.burst
        p.finish_time = time
        compute_metrics(p)
        add_gantt(gantt, p.pid, start, time)
        done += 1
```

```
    print_gantt(gantt)

    print_metrics(procs)



# ================= SRT
================= #


def srt(processes):

    procs = clone_processes(processes)

    time = 0

    done = 0

    gantt = []


    current = None

    start = 0


    while done < len(procs):

        ready = [p for p in procs if p.arrival <= time
and p.rem > 0]


        if not ready:

            if current is not None:

                add_gantt(gantt, current.pid if current
!= "IDLE" else "IDLE", start, time)

            current = "IDLE"

            start = time

            time += 1

            continue

        p = min(ready, key=lambda x: (x.rem,
x.arrival, x.pid))


        if current != p:

            if current is not None:

                add_gantt(gantt, current.pid if current
!= "IDLE" else "IDLE", start, time)

            current = p

            start = time


        p.rem -= 1

        time += 1


        if p.rem == 0:

            p.finish_time = time

            compute_metrics(p)

            done += 1


    if current is not None:

        add_gantt(gantt, current.pid if current !=
"IDLE" else "IDLE", start, time)


    print_gantt(gantt)

    print_metrics(procs)



# ================= ROUND ROBIN
================= #
```

```
def round_robin(processes, tq):

    procs = sorted(clone_processes(processes),
key=lambda x: (x.arrival, x.pid))

    time = 0

    i = 0

    done = 0


    queue = deque()

    gantt = []


    while done < len(procs):

        while i < len(procs) and procs[i].arrival <=
time:

            queue.append(procs[i])

            i += 1


        if not queue:

            add_gantt(gantt, "IDLE", time,
procs[i].arrival)

            time = procs[i].arrival

            continue


        p = queue.popleft()

        start = time


        run = min(tq, p.rem)

        time += run

        p.rem -= run

        add_gantt(gantt, p.pid, start, time)

        while i < len(procs) and procs[i].arrival <=
time:

            queue.append(procs[i])

            i += 1


        if p.rem > 0:

            queue.append(p)

        else:

            p.finish_time = time

            compute_metrics(p)

            done += 1


    print_gantt(gantt)

    print_metrics(procs)



# ================= PRIORITY NON-
PREEMPTIVE ================= #


def priority_np(processes, mode):

    procs = clone_processes(processes)

    time = 0

    done = 0

    gantt = []


    while done < len(procs):

        ready = [p for p in procs if p.arrival <= time
and p.finish_time is None]
```

```
        if not ready:

            next_arrival = min(p.arrival for p in procs
if p.finish_time is None)

            add_gantt(gantt, "IDLE", time,
next_arrival)

            time = next_arrival

            continue



        if mode == '2':  # higher number = higher
priority

            p = max(ready, key=lambda x: (x.priority,
-x.arrival, -int(x.pid[1:])))

        else:  # lower number = higher priority

            p = min(ready, key=lambda x: (x.priority,
x.arrival, x.pid))



        start = time

        time += p.burst

        p.finish_time = time

        compute_metrics(p)

        add_gantt(gantt, p.pid, start, time)

        done += 1



    print_gantt(gantt)

    print_metrics(procs)



# ================= PRIORITY + RR
================= #
```

```
def priority_rr(processes, tq, mode):

    procs = sorted(clone_processes(processes),
key=lambda x: (x.arrival, x.pid))

    time = 0

    i = 0

    done = 0



    queues = {}

    gantt = []



    def get_highest_priority(active):

        if mode == '2':

            return max(active)

        return min(active)



    while done < len(procs):

        while i < len(procs) and procs[i].arrival <=
time:

            p = procs[i]

            queues.setdefault(p.priority,
deque()).append(p)

            i += 1



        active = [k for k in queues if queues[k]]



        if not active:

            add_gantt(gantt, "IDLE", time,
procs[i].arrival)

            time = procs[i].arrival

            continue
```

```
        highest = get_highest_priority(active)

        queue = queues[highest]

        p = queue.popleft()


        start = time

        run = min(tq, p.rem)

        time += run

        p.rem -= run

        add_gantt(gantt, p.pid, start, time)


        while i < len(procs) and procs[i].arrival <=
time:

            new_p = procs[i]

            queues.setdefault(new_p.priority,
deque()).append(new_p)

            i += 1


        if p.rem > 0:

            queue.append(p)

        else:

            p.finish_time = time

            compute_metrics(p)

            done += 1


    print_gantt(gantt)

    print_metrics(procs)
```

```
# ================= MAIN
================= #


def main():

    while True:

        print("\n--- CPU Scheduling Simulator ---")

        print("1. FCFS")

        print("2. SJF (Non-Preemptive)")

        print("3. SRT (Preemptive)")

        print("4. Round Robin")

        print("5. Priority (Non-Preemptive)")

        print("6. Priority with RR")

        print("7. Exit")




        ch = input("Choice: ")

        if ch == '7':

            break


        if ch not in ['1','2','3','4','5','6','7']:

                print("Invalid menu choice.")

                continue


        n = int(input("Number of processes: "))

        processes = []


        priority_mode = None
```

```
    if ch in ['5', '6']:
        print("\nPriority Rule:")
        print("1. Lower number = higher priority")
        print("2. Higher number = higher priority")
        priority_mode = input("Choose mode (1
or 2): ")

    for i in range(n):
        pid = f"P{i+1}"
        arr = int(input(f"Arrival for {pid}: "))
        burst = int(input(f"Burst for {pid}: "))
        pri = 0

        if ch in ['5', '6']:
            pri = int(input(f"Priority for {pid}: "))

        processes.append(Process(pid, arr,
burst, pri))

    if ch == '1':
        fcfs(processes)
    elif ch == '2':
        sjf(processes)
    elif ch == '3':
        srt(processes)
    elif ch == '4':
        tq = int(input("Time Quantum: "))
        round_robin(processes, tq)
    elif ch == '5':
        priority_np(processes, priority_mode)
    elif ch == '6':
        tq = int(input("Time Quantum: "))
        priority_rr(processes, tq, priority_mode)

if __name__ == "__main__":
    main()
```

2.2 First-Come, First-Served (FCFS)

works by executing processes strictly in the order they arrive in the ready queue. The process that arrives first is given the CPU first, and it continues running until it is fully completed before the next process is allowed to execute. This scheduling method is non-preemptive, meaning once a process starts, it cannot be interrupted until it finishes.

One of its main advantages is its simplicity, as it is very easy to implement and understand. It does not require complex sorting or decision-making, which makes it straightforward to use in basic systems. However, its main limitation is the “Convoy Effect,” where a long-running process that arrives early forces all shorter processes

behind it to wait. This often leads to increased average waiting time and reduced overall system efficiency.

Input: P1 (Arrival: 0, Burst: 2), P2 (Arrival: 1, Burst: 3), P3 (Arrival: 2, Burst: 2)

Gantt Chart:

P1	P2	P3
0–2	2-5	5-7

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	2	0	2
P2	1	3	1	4
P3	2	2	3	5
Average			**1.33**	**3.67**

```
--- CPU Scheduling Simulator ---
1. FCFS
2. SJF (Non-Preemptive)
3. SRT (Preemptive)
4. Round Robin
5. Priority (Non-Preemptive)
6. Priority with RR
7. Exit
Choice: 1
Number of processes: 3
Arrival for P1: 0
Burst for P1: 2
Arrival for P2: 1
Burst for P2: 3
Arrival for P3: 2
Burst for P3: 2

Gantt Chart:
| P1 (0-2) | P2 (2-5) | P3 (5-7) |

PID     WT      TAT
P1      0       2
P2      1       4
P3      3       5

Average WT: 1.33
Average TAT: 3.67
```

Test Case 2

Input: P1 (Arrival: 0, Burst: 2), P2 (Arrival: 1, Burst: 2), P3 (Arrival: 2, Burst: 2)

Gantt Chart:

P1	P2	P3
0–2	2–4	4-6

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	2	0	2
P2	1	2	1	3
P3	2	2	2	4
Average			**1.00**	**3.00**

```
Choice: 1
Number of processes: 3
Arrival for P1: 0
Burst for P1: 2
Arrival for P2: 1
Burst for P2: 2
Arrival for P3: 2
Burst for P3: 2

Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) |

PID     WT      TAT
P1      0       2
P2      1       3
P3      2       4

Average WT: 1.00
Average TAT: 3.00
```

2.3 Shortest Job First (SJF) – Non-Preemptive

Operating as a non-preemptive algorithm, Shortest Job First (SJF) selects the available process with the smallest burst time at each decision point and executes it until completion without any interruption. Its main advantage is its efficiency in theory, since it can achieve the lowest possible average waiting time for a given set of processes, assuming that burst times are known beforehand. However, it also has practical drawbacks. One major issue is starvation, where longer processes may be continuously delayed if shorter processes keep arriving. In addition, SJF depends on accurate

knowledge of burst times, which makes it difficult to use in real systems where process durations are often unpredictable.

Test Case 1

Input: P1 (Arrival: 0, Burst: 3), P2 (Arrival: 1, Burst: 1), P3 (Arrival: 2, Burst: 4)

Gantt Chart:

P1	P3	P2
0–3	3-4	4-8

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	0	3
P2	1	1	2	3
P3	2	4	2	6
Average			**1.33**	**4.00**

```
Gantt Chart:
| P1 (0-3) | P2 (3-4) | P3 (4-8) |

PID     WT      TAT
P1      0       3
P2      2       3
P3      2       6

Average WT: 1.33
Average TAT: 4.00
```

Test Case 2

Input: P1 (Arrival: 0, Burst: 3), P2 (Arrival: 2, Burst: 4), P3 (Arrival: 4, Burst: 1)

Gantt Chart:

P1	P3	P2
0–3	3-4	4-8

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	0	3
P2	2	4	3	7
P3	4	1	1	2
Average			**1.33**	**4.00**

```
Gantt Chart:
| P1 (0-3) | P3 (3-4) | P2 (4-8) |

PID     WT      TAT
P1      0       3
P2      3       7
P3      1       2

Average WT: 1.33
Average TAT: 4.00
```

2.4 Shortest Remaining Time (SRT) – Preemptive

As the preemptive counterpart to Shortest Job First (SJF), the Shortest Remaining Time (SRT) algorithm constantly evaluates the CPU queue to ensure the most efficient process is running. Whenever a new process enters the ready queue, the scheduler compares its burst time against the remaining time of the active process; if the newcomer is shorter, the current process is preempted and the new one takes over. This approach is highly responsive and typically yields superior average waiting times in dynamic environments where processes arrive at different intervals. However, this efficiency comes at a cost, as

frequent context switching introduces significant system overhead. Additionally, SRT still faces the risk of process starvation for longer tasks and requires increased implementation complexity to track and update remaining burst times in real-time.

Test Case 1

Input: P1 (Arrival: 0, Burst: 3), P2 (Arrival: 1, Burst: 4), P3 (Arrival: 2, Burst: 2)

Gantt Chart:

P1	P3	P2
0–3	3-5	5-9

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	6	0	3
P2	1	4	4	8
P3	2	2	1	3
Average			**1.67**	**4.67**

```
=== Shortest Remaining Time (Preemptive) ===
Gantt Chart: | P1 (0-1) | P2 (1-2) | P3 (2-4) | P2 (4-7) | P1 (7-12) |

----------------------------------------
PID     WT      TAT
P1      6       12
P2      2       6
P3      0       2

Average WT: 2.67
Average TAT: 6.67
----------------------------------------
```

Test Case 2

Input: P1 (Arrival: 0, Burst: 2), P2 (Arrival: 1, Burst: 4), P3 (Arrival: 3, Burst: 5)

Gantt Chart:

P1	P2	P3

0–2	2-6	6-11

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	2	2	7
P2	1	4	0	2
P3	3	5	4	8
Average			**2.00**	**5.67**

```
Gantt Chart:
| P1 (0-2) | P2 (2-6) | P3 (6-11) |

PID     WT      TAT
P1      0       2
P2      1       5
P3      5       10

Average WT: 2.00
Average TAT: 5.67
```

2.5 Round Robin (RR)

Round Robin is a preemptive scheduling algorithm designed for fairness and responsiveness, especially in time-sharing systems. It operates by assigning each process a fixed time interval known as a **Time Quantum (TQ)**. The CPU cycles through the ready queue, granting each process one TQ of execution time; if a process does not finish within its allotted slice, it is preempted and moved to the back of the queue to await its next turn. This ensures that no single task can monopolize the CPU, providing excellent response times for all users. However, the algorithm's efficiency is heavily dependent on the size of the Time Quantum. If the TQ is too small, the system suffers from excessive context-switching overhead, whereas an overly large TQ causes the algorithm to degrade into First-Come, First-Served (FCFS) behavior, potentially leading to long wait times for shorter processes.

Test Case 1 (Time Quantum = 2)

Input: P1 (Arrival: 0, Burst: 3), P2 (Arrival: 1, Burst: 4), P3 (Arrival: 2, Burst: 2)

Gantt Chart:

P1	P2	P3	P1	P2
0–2	2–4	4–6	6-7	7-9

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	4	7
P2	1	4	4	8
P3	2	2	2	4
Average			**3.33**	**6.33**

```
Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) | P1 (6-7) | P2 (7-9) |

PID     WT      TAT
P1      4       7
P2      4       8
P3      2       4

Average WT: 3.33
Average TAT: 6.33
```

Test Case 2 (Time Quantum = 2)

Input: P1 (Arrival: 0, Burst: 2), P2 (Arrival: 0, Burst: 3), P3 (Arrival: 0, Burst: 4)

Gantt Chart:

P1	P2	P3	P2	P3
0–2	2–4	4–6	6-7	7–9

Computed Metrics:

Process	Arrival Time	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	2	0	2
P2	0	3	4	7
P3	0	4	5	9
Average			**3.00**	**6.00**

```
Time Quantum: 2

Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) | P2 (6-7) | P3 (7-9) |

PID     WT      TAT
P1      0       2
P2      4       7
P3      5       9

Average WT: 3.00
Average TAT: 6.00
```

2.6 Priority Scheduling (Non-Preemptive)

Non-Preemptive Priority Scheduling operates by assigning a numerical priority value to every process, where a lower number typically signifies a higher priority (e.g., priority 1 precedes priority 3). Whenever the CPU becomes idle, the scheduler selects the highest-priority process from the ready queue and executes it to completion without interruption.

This method provides the operating system with significant control, ensuring that critical system tasks are prioritized over less urgent background jobs. However, the primary drawback is the risk of "starvation," as low-priority processes may remain in the queue indefinitely if higher-priority tasks continue to arrive. While "aging" a technique that gradually increases a process's priority based on its wait time is a common solution to this issue, it is often omitted in basic implementations for the sake of simplicity.

Test Case 1

Input: P1 (Arrival: 0, Burst: 3, Priority: 3), P2 (Arrival: 2, Burst: 1, Priority: 2), P3 (Arrival: 1, Burst: 2, Priority: 1)

Gantt Chart:

P1	P2	P3
0–3	3-5	5-6

Computed Metrics:

Process	Arrival Time	Burst Time	Priority	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	3	0	3
P2	2	1	2	3	4
P3	1	2	1	2	4
Average				**1.67**	**3.67**

```
Gantt Chart:
| P1 (0-3) | P3 (3-5) | P2 (5-6) |

PID     WT      TAT
P1      0       3
P2      3       4
P3      2       4

Average WT: 1.67
Average TAT: 3.67
```

Test Case 2

Input: P1 (Arrival: 0, Burst: 3, Priority: 2), P2 (Arrival: 1, Burst: 2, Priority: 3), P3 (Arrival: 2, Burst: 2, Priority: 2)

Gantt Chart:

P1	P3	P2
0–3	3–5	5–7

Computed Metrics:

Process	Arrival Time	Burst Time	Priority	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	2	0	3
P2	1	2	3	4	6
P3	2	2	2	1	3
Average				**1.67**	**4.00**

```
=== Priority Scheduling (Non-Preemptive) ===
Gantt Chart: | P1 (0-3) | P3 (3-5) | P2 (5-10) |

-----------------------------------------
PID     WT      TAT
P1      0       3
P2      4       9
P3      1       3

Average WT: 1.67
Average TAT: 5.00
-----------------------------------------
```

2.7 Priority Scheduling with Round Robin

Priority with Round Robin is a hybrid and preemptive scheduling algorithm that combines the structure of priority scheduling with the fairness of Round Robin. In this method, processes are first grouped according to their priority levels, where higher

priority processes are always given CPU access before lower ones. However, when several processes have the same priority, the scheduler applies Round Robin by assigning a fixed time quantum to each process so that CPU time is shared fairly among them.

This combined approach helps ensure that important tasks are executed first while still preventing processes of equal priority from being ignored or delayed for too long. Even so, the method can be more complex to implement because the system needs to manage multiple priority queues at the same time and also keep track of time slices for Round Robin execution within each group, which can increase overall overhead.

Test Case 1 (Time Quantum = 2)

Input: P1 (Arrival: 2, Burst: 1, Priority: 3), P2 (Arrival: 1, Burst: 2, Priority: 2), P3 (Arrival: 0, Burst: 3, Priority: 1)

Gantt Chart:

P3	P2	P1
0–3	3-5	5-6

Computed Metrics:

Process	Arrival Time	Burst Time	Priority	Waiting Time (WT)	Turnaround Time (TAT)
P1	2	1	3	3	4
P2	1	2	2	2	4
P3	0	3	1	0	3
Average				**1.67**	**3.67**

```
Gantt Chart:
| P3 (0-3) | P2 (3-5) | P1 (5-6) |

PID     WT      TAT
P1      3       4
P2      2       4
P3      0       3

Average WT: 1.67
Average TAT: 3.67
```

Test Case 2 (Time Quantum = 2)

Input: P1 (Arrival: 0, Burst: 3, Priority: 2), P2 (Arrival: 0, Burst: 4, Priority: 2), P3 (Arrival: 0, Burst: 4, Priority: 1)

Gantt Chart:

P3	P1	P2	P1	P2
0–4	4-6	6-8	8-9	9-11

Computed Metrics:

Process	Arrival Time	Burst Time	Priority	Waiting Time (WT)	Turnaround Time (TAT)
P1	0	3	2	6	9
P2	0	4	2	7	10
P3	0	4	1	0	4
Average				**4.33**	**8.00**

```
Time Quantum: 2

Gantt Chart:
| P3 (0-4) | P1 (4-6) | P2 (6-8) | P1 (8-9) | P2 (9-11) |

PID     WT      TAT
P1      6       9
P2      7       11
P3      0       4

Average WT: 4.33
Average TAT: 8.00
```

2.8 Comparison of Scheduling Algorithms

The table below summarizes how each algorithm performed across key metrics in the test cases above:

Algorithm	Type	Avg WT (TC1)	Avg TAT (TC1)	Starvation Risk	Complexity
FCFS	Non-Preemptive	1.33	3.67	None	Low
SJF	Non-Preemptive	1.33	4.00	High	Medium
SRT	Preemptive	1.67	4.67	High	Medium
Round Robin	Preemptive	3.33	6.33	None	Medium
Priority	Non-Preemptive	1.67	3.67	High	Medium
Priority + RR	Preemptive	1.67	3.67	Medium	High

3. Summary / Takeaways

This case study helped me gain a more practical understanding of CPU scheduling concepts that initially seemed simple during lectures. One of the main realizations is that there is no single scheduling method that performs best in all situations. Each algorithm has its own strengths and weaknesses depending on the goal being optimized.

From the simulation results, Shortest Job First (SJF) usually achieved the lowest average waiting time, but this assumes that burst times are already known, which is rarely the case in real operating systems. First-Come, First-Served (FCFS) was easy to implement but clearly showed poor performance in some cases due to the Convoy Effect, where a long process at the front delays all the others behind it.

The preemptive methods, such as Shortest Remaining Time (SRT) and Round Robin, handled dynamic process arrivals more effectively. However, this improvement comes with the drawback of increased context switching, which adds overhead to the system. Round Robin, in particular, stood out in terms of fairness since it ensures that every process gets CPU time within a fixed interval, even if it does not always provide the best average waiting time. Priority scheduling, on the other hand, is useful when certain tasks need to be executed first, but it can result in starvation if lower-priority processes are not properly managed. The combined Priority with Round Robin approach helps reduce this issue by balancing fairness and priority, although it is more complicated to implement and analyze.

One of the most difficult parts of building the simulator was correctly handling preemptive execution, especially when processes are interrupted and later resumed. Keeping track of remaining burst times and accurately updating system time required careful testing and multiple manual checks to ensure the results, including the Gantt chart and performance metrics, were correct.

Overall, this study showed why real operating systems do not depend on just one scheduling algorithm. Modern schedulers like Linux’s Completely Fair Scheduler (CFS) or Windows Multilevel Feedback Queue use a combination of techniques and adapt based on system behavior. Learning these basic algorithms first makes it much easier to understand how those advanced scheduling systems work.

4. References

GeeksforGeeks. (2023). *CPU Scheduling in Operating Systems*. https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.